Embedded Systems (M233)
Functions and Montages

Every Arduino function the course uses: what it does in the hardware, what it returns, how it fails. Every circuit the past papers draw: the schematic, the code that drives it, and the questions asked on it

Built from Pr. Aissam Berrahou's two lecture decks, the six labs and the four past midterms

Every code output in this document was run in the course simulator; 16-bit overflow cases follow the AVR rules

How to read this document

The midterm has the same five kinds of exercise every year: a **QCM** or true/false, a **types table**, a **montage à compléter** (finish or diagnose a wiring), **code to analyse, debug or trace**, and an **FSM**. All of them come down to two skills: knowing exactly what each function does to a pin, and reading a circuit as voltages. Part 1 is one card per function. Part 2 is the C++ that the trace questions test. Part 3 is every circuit in the papers, each drawn, coded and questioned. Part 4 is the code patterns that sit on top of those circuits, with their real outputs. Part 5 is a 32-question quiz with a justified answer key. The FSM method itself is in Part G of the complete guide.

French exam vocabulary. *montage* = wiring; *câbler* = to wire; *broche* = pin; *masse* = GND; *résistance de tirage* = pull-up/pull-down resistor; *entrée flottante* = floating input; *rebond* = bounce; *état* = state; *rapport cyclique* = duty cycle; *anode/cathode*; *actif à l'état bas* = active LOW.

1 The Board You Answer From

Item	Value and why it matters
Chip, clock	ATmega328P at 16 MHz; logic levels 0 V and 5 V
Digital pins	D0–D13. **D0 = RX, D1 = TX**: used by Serial and by uploading, keep them free
PWM pins (~)	3, 5, 6, 9, 10, 11 only; about 490 Hz (980 Hz on 5 and 6)
Analog inputs	A0–A5, 10-bit ADC: 0–1023, about 4.9 mV per step. A0–A5 also work as digital pins D14–D19
On-board LED	`LED_BUILTIN` = pin 13
Current per pin	20 mA recommended, 40 mA absolute maximum, 200 mA for the whole chip
Memory	Flash 32 KB (program, 0.5 KB bootloader), SRAM 2 KB (variables, stack: runs out fast), EEPROM 1 KB (survives power off)
Power	about 3 mA active at 4 MHz, 3 V; about 0.1 µA in sleep (slide values)

Key insight. A pin is either an *output* (the chip forces 0 V or 5 V onto the wire) or an *input* (the chip only looks at the wire and must be told what voltage is there by the circuit). Almost every montage question is about which of the

two it is and what fixes the voltage.

2 Part 1: The Functions, One Card Each

2.1 `pinMode`, `digitalWrite`, `digitalRead`

Function: `pinMode(pin, mode)`

`mode` is `OUTPUT`, `INPUT` or `INPUT_PULLUP`. Called once in `setup()`. It sets the pin's direction bit (register DDRx) and, for `INPUT_PULLUP`, switches on an internal 20–50 kΩ resistor to 5 V (register PORTx bit). Returns nothing.

- At reset every pin is an `INPUT`. Right after `pinMode(p, OUTPUT)` the pin is **LOW**.
- `analogWrite` sets `OUTPUT` by itself; `analogRead` needs no `pinMode`.

Function: `digitalWrite(pin, value)`

`value` is `HIGH` (1) or `LOW` (0). On an `OUTPUT` pin it drives 5 V or 0 V. On an `INPUT` pin it does something else: `HIGH` turns the internal pull-up on, `LOW` turns it off (this is how `INPUT_PULLUP` worked before it existed). Takes a few microseconds.

Function: `digitalRead(pin)`

Returns `int`: `HIGH` (1) if the pin is above about 3 V, `LOW` (0) below about 1.5 V. In between, and on a pin that nothing drives (**floating**), the result is random: it follows noise, your finger, the neighbouring wire.

`pinMode`	`digitalWrite`	What the pin is electrically
`OUTPUT`	`HIGH`	driven to 5 V, can source about 20 mA
`OUTPUT`	`LOW`	driven to 0 V, can sink about 20 mA
`INPUT`	(none or `LOW`)	high impedance: floating unless the circuit fixes it
`INPUT`	`HIGH`	internal pull-up on: reads HIGH when nothing pulls it down
`INPUT_PULLUP`	(none)	same as the row above

2.2 `analogRead`, `analogWrite`, `map`, `constrain`

Function: `analogRead(pin)`

Only on A0–A5. Returns `int` 0..1023 proportional to the voltage, with 5 V as reference:

$$\text{value} \approx V_{\text{in}} \times \frac{1023}{5}, \qquad V_{\text{in}} \approx \text{value} \times \frac{5}{1023}.$$

0 V gives 0, 2.5 V gives about 512, 5 V gives 1023. One conversion takes about 100 µs. An unconnected analog pin returns drifting noise. Values are integers: 3.3 V gives 675.

Function: `analogWrite(pin, value)`

Only on PWM pins 3, 5, 6, 9, 10, 11; `value` 0..255. It does **not** output an analog voltage: a hardware timer switches the pin between 0 V and 5 V about 490 times a second, HIGH for a

fraction value/255 of each period (the **duty cycle**). An LED or a motor responds to the average:

$$\text{duty} = \frac{\text{value}}{255}, \qquad \bar{V} = 5 \times \frac{\text{value}}{255}.$$

128 gives 50 %, about 2.5 V average; 64 gives 25 %, about 1.25 V. 0 is always LOW, 255 always HIGH. Traps: on a non-PWM pin, values below 128 give LOW and the rest HIGH; a value above 255 is not clipped but truncated to its low byte (300 becomes 44), so constrain first.

Function: `map(x, inLow, inHigh, outLow, outHigh)` and `constrain(x, a, b)`

`map` rescales with integer arithmetic, truncating:

$$\text{map}(x) = \frac{(x - \text{inLow})(\text{outHigh} - \text{outLow})}{\text{inHigh} - \text{inLow}} + \text{outLow}.$$

`map(512, 0, 1023, 0, 255)` is 127, not 127.6; `map(1023, 0, 1023, 0, 255)` is 255. It does not clip: an input outside the range gives an output outside the range. `constrain(x, a, b)` clips: `constrain(300, 0, 255)` is 255. Reversing the output range inverts: `map(x, 0, 1023, 255, 0)`.

2.3 Time: `delay`, `millis`, `micros`

Function: `delay(ms)` and `delayMicroseconds(us)`

Stops the program for that many milliseconds (microseconds). Nothing else in `loop()` runs: a button pressed and released during `delay(5000)` is never seen. `millis()` keeps counting during a delay.

Function: `millis()` and `micros()`

Return `unsigned long`: milliseconds (microseconds) since reset. `millis()` wraps to 0 after 2^{32} ms $\approx$ 49.7 days; `micros()` has a 4 µs resolution and wraps after about 71.6 minutes. The only correct timing test is the subtraction

```
if (millis() - previous >= interval) { previous += interval; /* act */ }
```

which stays right across the wrap because unsigned subtraction is modular. `millis() >= previous + interval` breaks at the wrap. Storing `millis()` in an `int` breaks after 32.767 s (it goes negative). `previous += interval` keeps a steady period; `previous = millis()` lets the small lateness of each loop accumulate (drift).

2.4 Serial

Function: `Serial.begin(baud)`, `Serial.end()`

Opens the UART on D0 (RX) and D1 (TX) at `baud` bits per second, format 8N1: 10 bits on the wire per byte, so 9600 baud is 960 bytes/s, about 1 ms per character. From then on D0 and D1 are not usable for LEDs or buttons; `Serial.end()` frees them. Both boards must use the same baud.

Function: `print`, `println`, `write`

`print(x)` sends the *text* of `x`; `println` adds `"\r\n"`. `write(b)` sends the raw byte. So `print(66)` shows `66` and `write(66)` shows `B`. Floats print with 2 decimals (`3.14159` shows `3.14`); `print(x, 4)` gives 4. Bases: `println(255, HEX)` shows `FF`, `println(5, BIN)` shows `101`. Arithmetic on characters is integer: `println('A' + 2)` shows 67, but `char c = 'A' + 2; println(c)` shows `C`.

Function: `available`, `read`, `peek`

`available()` returns how many received bytes wait in the 64-byte buffer. `read()` returns the next byte as an `int` and removes it, or `-1` if the buffer is empty. `peek()` returns it without removing it. Received digits are characters: typing `5` makes `read()` return 53 (`'5'`); the digit value is `c - '0'`.

Function: `parseInt`, `parseFloat`, `readStringUntil`, `readBytes`

`parseInt()` skips characters that cannot start a number, reads the digits (and a leading `-`), and stops at the first other character. If nothing arrives within the timeout (1000 ms) it returns **0**. `readStringUntil('\n')` returns a `String` without the terminator. `readBytes(buf, n)` fills `buf` with up to `n` bytes.

Warning / Gotcha: The newline bug, run in the simulator

```
void loop() {
  if (Serial.available() > 0) {
    int n = Serial.parseInt();
    Serial.print("got "); Serial.println(n);
  }
}
```

Typing `5` with the Serial Monitor set to "Newline" sends `5\n`. Output: `got 5`, then one second later `got 0`. The first `parseInt` stops at `\n` and leaves it in the buffer; the next loop sees `available() > 0`, `parseInt` finds no digit, waits the 1 s timeout and returns 0. Fixes: set the monitor to "No line ending", or empty the buffer after reading (`while (Serial.available()) Serial.read();`), or use `readStringUntil('\n')` then `toInt()`.

2.5 EEPROM

Function: `EEPROM.read`, `write`, `update`, `get`, `put` (`#include <EEPROM.h>`)

1024 bytes at addresses 0..1023, kept without power. `read(addr)` returns one byte (a never-written cell reads 255). `write(addr, v)` stores one byte (`v` is truncated to 0..255) and takes about 3.3 ms. `update(addr, v)` writes only if the value differs. `put(addr, x)` and `get(addr, x)` store and load any type, using `sizeof(x)` bytes from `addr`. Each cell survives about 100 000 writes: writing on every pass of `loop()` (one write per 3.3 ms) wears it out in $100\,000 \times 3.3$ ms ≈ 5.5 minutes. Write on change only.

2.6 `tone` and buzzers

Function: `tone(pin, freq[, ms])`, `noTone(pin)`

Square wave at `freq` Hz, 50 % duty, on any pin, for `ms` milliseconds or until `noTone`. One tone at a time. A **passive** buzzer needs it; an **active** buzzer has its own oscillator and just needs `digitalWrite(pin, HIGH)` (the course's alarms use the active kind).

3 Part 2: The C++ the Trace Questions Test

Type (Uno)	Bytes	Range	Use
`bool`	1	`true`/`false`	flags
`char`	1	−128..127	characters (`'A'` = 65, `'0'` = 48)
`byte`, `uint8_t`	1	0..255	pins, small counters, EEPROM cells
`int`	**2**	−32 768..32 767	general integers
`unsigned int`	2	0..65 535	positive counters
`long`	4	about $\pm 2.1 \times 10^9$	large numbers
`unsigned long`	4	0..4 294 967 295	`millis()`, `micros()`
`float`, `double`	4	$\pm 3.4 \times 10^{38}$, 6–7 digits	decimals (`double` = `float` on Uno, slow)

Worked example: Arithmetic traps, with the value the Uno produces

Code	Result	Why
`int x = 32767; x++;`	−32 768	16-bit `int` wraps
`unsigned long t = 60 * 1000;`	4 294 961 760	`60*1000` is computed as `int`: overflows to −5536, then converted
`unsigned long t = 60UL * 1000;`	60 000	the `UL` makes the product `unsigned long`
`byte b = 250; b += 10;`	4	260 mod 256
`float f = 5 / 2;`	2.00	integer division first, conversion after
`7 / 2.0`	3.50	one operand is `float`
`-7 / 2, 7 % 3`	−3, 1	division truncates toward 0
`'A' + 2`	67	characters are small integers
`int a = ++i;` with `i = 5`	`a = 6, i = 6`	prefix: increment, then use
`int b = i++;` then	`b = 6, i = 7`	postfix: use, then increment
`sizeof(arr) / sizeof(arr[0])`	element count	`sizeof` counts bytes

3.1 Scope and constants

A **global** variable (declared outside any function) lives for the whole program and every function sees it. A **local** variable is created at each call and forgotten at the end. A **static local** is initialised once and keeps its value between calls:

```
int counter() { static int n = 0; n++; return n; }   // counter(); counter(); counter() returns 3
```

Without `static`, it would return 1 every time. Constants: `#define LED 13` is text replacement (no type, no semicolon); `const int LED = 13;` is typed and checked; `constexpr` is computed at compile time. `volatile` marks a variable changed outside the normal flow (an interrupt routine), so the compiler re-reads it every time.

3.2 References, pointers, arrays

```
void byValue(int x)  { x = x * 2; }     // works on a copy
void byRef(int &x)   { x = x * 2; }     // x is another name for the caller's variable
void byPtr(int *p)   { *p = *p + 1; }   // p holds the address; *p is the variable
int v = 10;
byValue(v);  // v = 10
byRef(v);    // v = 20
byPtr(&v);   // v = 21
int arr[] = {4, 8, 15, 16};
int *p = arr;  p++;     // an array name is the address of element 0; p++ moves one element
Serial.println(*p);     // 8
```

`&` in a declaration (`int &x`) makes a reference; `&v` in an expression takes an address; `*p` in an expression reads or writes what `p` points to. A function that must change a `struct` takes it by reference: `void press(Button &b)`.

3.3 `struct`, `enum`, `switch`

```
struct Button { int pin; int lastState; unsigned long lastChange; };
Button buttons[] = { {2, HIGH, 0}, {3, HIGH, 0} };   // array of structs
for (int i = 0; i < 2; i++) pinMode(buttons[i].pin, INPUT_PULLUP);

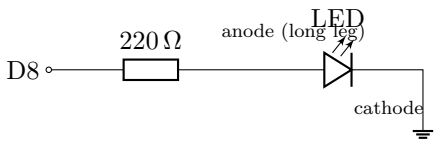
enum Light { RED, GREEN, YELLOW };            // RED = 0, GREEN = 1, YELLOW = 2; converts to int
enum class State : uint8_t { IDLE, ARMED, ALARM };   // scoped: State::IDLE; no implicit int
State state = State::IDLE;

switch (state) {
  case State::IDLE:  /* ... */ break;
  case State::ARMED: /* ... */ break;      // without break, execution falls into the next case
  case State::ALARM: /* ... */ break;
}
```

4 Part 3: The Montages

Read every circuit as voltages: an output pin is a 0 V or 5 V source; an input pin draws no current and only reports the voltage the circuit puts on it. Series resistors limit current; pull resistors fix a default voltage.

4.1 M1. LED, active HIGH (the course's standard)




```
pinMode(8, OUTPUT);  digitalWrite(8, HIGH);   // ON
```

Current: $I = \frac{5-V_F}{R}$ with $V_F \approx 2$ V (red) to 3 V (blue, white). With 220 Ω and a red LED, $I = 3/220 =$ 13.6 mA, under the 20 mA limit. Choosing R: $R = \frac{5-V_F}{I}$, then the next standard value *above* (2 V at

15 mA: 200 Ω, take 220 Ω). The slides wire pin → 220 Ω → anode, cathode → GND rail; the resistor can be on either side of the LED.

Exam questions on it

Q1. The LED is put in backwards. What happens?

Answer. It does not light and nothing is damaged: a diode blocks reverse current (5 V is below its reverse breakdown).

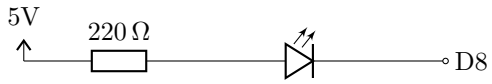
Q2. The resistor is replaced by a wire.

Answer. The current is limited only by the pin and the LED: far above 40 mA; the LED or the pin is damaged over time. Never connect an LED without a series resistor.

Q3. Three LEDs share one resistor in their common GND wire.

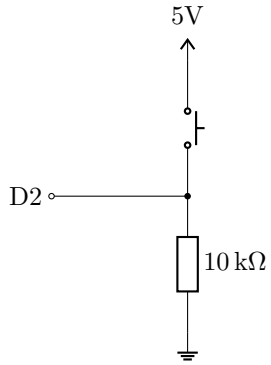
Answer. They share one current: brightness changes with how many are on, and LEDs of different V_F steal current from each other. One resistor per LED.

4.2 M2. LED, active LOW (current sink)



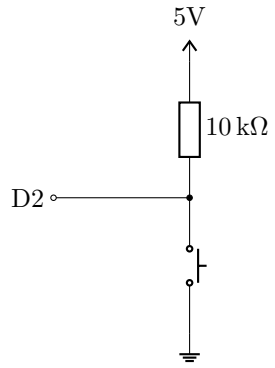
The LED lights when D8 is **LOW** (current flows from 5 V into the pin) and is off when D8 is HIGH (no voltage across it). Trap: `pinMode(8, OUTPUT)` alone already lights it, because an output starts LOW. The on-board LED and most relay modules work the other way round; read the schematic, not the code.

4.3 M3. Push button with external pull-down



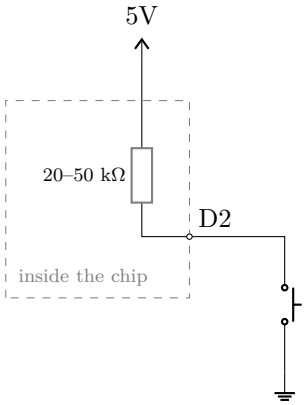
`pinMode(2, INPUT)`. Released: the 10 kΩ connects D2 to GND, reads **LOW**. Pressed: D2 is connected to 5 V, reads **HIGH**; current through the resistor $5/10\,000 = 0.5$ mA. Active HIGH.

4.4 M4. Push button with external pull-up



`pinMode(2, INPUT)`. Released: HIGH. Pressed: LOW. Active LOW.

4.5 M5. Push button with the internal pull-up



`pinMode(2, INPUT_PULLUP)`: one wire from D2 to the button, one from the button to GND, no resistor. Released HIGH, pressed **LOW**. This is the course's default (`while (digitalRead(BTN) == HIGH) {}` waits for a press).

Exam questions on it

Q1. M3's wiring is used with `INPUT_PULLUP`.

Answer. Released, the internal pull-up (about 35 kΩ) and the external 10 kΩ to GND form a divider: $5 \times 10/(10 + 35) \approx 1.1$ V, an ambiguous level. Use one pull resistor, not two in opposition.

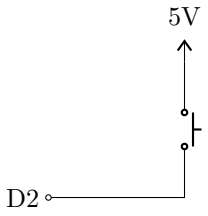
Q2. A student wires the button between 5 V and GND directly, with D2 on one side.

Answer. Pressing it short-circuits 5 V to GND: the board resets or the USB port cuts out. A button always goes through a pull resistor or to a pin, never straight across the supply.

Q3. Which value is read when pressed, in M3, M4, M5?

Answer. HIGH, LOW, LOW. The pull resistor fixes the released level; pressing connects the pin to the other rail.

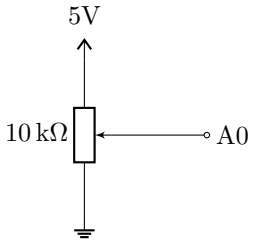
4.6 M6. The floating input (Lab "Floating problem")



`pinMode(2, INPUT)`, no resistor. Pressed: HIGH. Released: the pin is connected to nothing; it reads random values that change when you bring a hand close. Fix: add M3's pull-down, or rewire as M5.

Key insight. A pull resistor does two jobs: it gives the open switch a defined level, and when the switch closes it limits the current between the rails to a fraction of a milliampere.

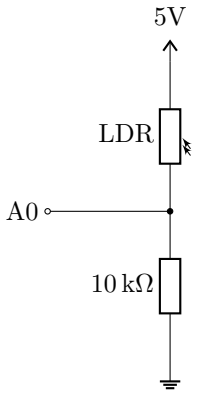
4.7 M7. Potentiometer on an analog input




```
int raw = analogRead(A0);                  // 0..1023
analogWrite(9, map(raw, 0, 1023, 0, 255)); // LED on PWM pin 9: brightness follows the knob
```

The wiper divides the 5 V: at the 5 V end `raw` = 1023, in the middle about 512, at the GND end 0. `raw / 4` is a cheap version of the same `map`. Trap: the wiper on a digital pin gives only HIGH or LOW; the LED on a non-PWM pin only switches on or off.

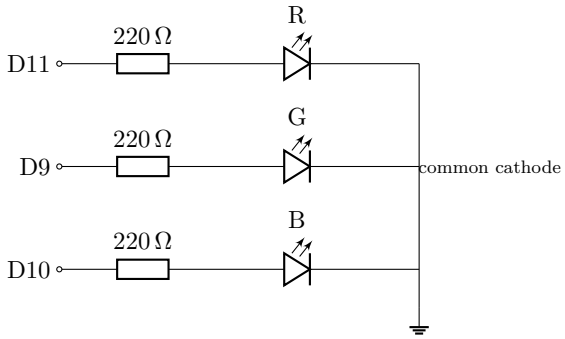
4.8 M8. Light sensor (LDR) as a voltage divider



$$V_{A0} = 5 \times \frac{10\,\text{k}\Omega}{R_{\text{LDR}} + 10\,\text{k}\Omega}.$$

Bright ($R_{\text{LDR}} \approx 1$ kΩ): 4.55 V, reading about 930. Dark (100 kΩ): 0.45 V, about 93. More light, higher reading. Swap the LDR and the resistor and the sense inverts. A threshold such as `if (analogRead(A0) < 300) digitalWrite(LAMP, HIGH);` turns a lamp on at dusk.

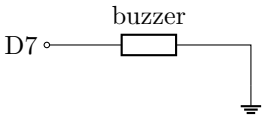
4.9 M9. RGB LED




```
const int Rpin = 11, Gpin = 9, Bpin = 10;          // all PWM
void setRGB(int r, int g, int b) { analogWrite(Rpin, r); analogWrite(Gpin, g); analogWrite(Bpin, b); }
setRGB(255, 0, 255);   // magenta;  setRGB(255, 255, 0) yellow;  setRGB(255, 255, 255) white
```

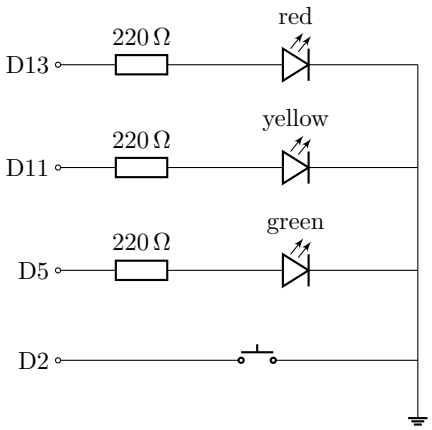
Each colour needs its own PWM pin and resistor. With a **common anode** LED the common leg goes to 5 V and the logic inverts: `analogWrite(Rpin, 255 - r)`; 0 means full red.

4.10 M10. Buzzer



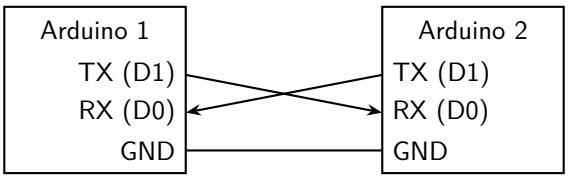
Active buzzer: `digitalWrite(7, HIGH)` sounds, `LOW` is silent. Passive buzzer: `tone(7, 2000)` for a 2 kHz tone, `noTone(7)` to stop. A plain HIGH on a passive buzzer only clicks once.

4.11 M11. Traffic light with a pedestrian button (Lab FSM)



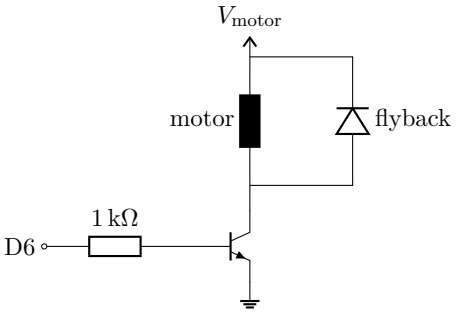
Three active-HIGH LEDs (pins and resistors from the slides), one button to GND with `INPUT_PULLUP`, all returns on the breadboard's GND rail, which must be joined to the Arduino's GND. The code is an FSM: one state per light, a timer per state with `millis()`, the button read without blocking.

4.12 M12. Two boards talking over Serial



TX goes to the other board's RX, both ways (crossed), and the two GNDs are joined: a voltage only means something relative to a shared ground. Same baud rate on both. Unplug these wires while uploading, since the USB link uses D0 and D1 too.

4.13 M13. Anything that needs more than 20 mA



A motor, relay or lamp is switched by a transistor: the pin supplies a few mA to the base through a resistor, the transistor carries the load current from its own supply. The diode across the coil absorbs the voltage spike when it switches off. Never power a motor from a pin.

4.14 Diagnosing a montage

Warning / Gotcha: The faults the papers plant, and the fix

- LED without series resistor: add 220–330 Ω. LED reversed: swap legs (long leg towards the + side).
- Button with no pull resistor on an `INPUT`: floating; add a pull resistor or use `INPUT_PULLUP`.
- Button across 5 V and GND: short circuit when pressed.
- Logic inverted in the code: `INPUT_PULLUP` reads LOW when pressed.
- LED or button on D0/D1 while Serial is used: conflict; move to D2–D13.
- `analogWrite` on a non-PWM pin (2, 4, 7, 8, 12, 13): only on/off. `analogRead` on a digital pin: not possible.
- Sensor or second board without a common GND: readings meaningless.
- Load above 20 mA on a pin: use a transistor. Several LEDs sharing one resistor: one resistor each.
- Missing `pinMode(p, OUTPUT)` before `digitalWrite`: the pin stays an input, the LED glows

faintly (pull-up current) or not at all.

5 Part 4: The Code Patterns on Top of the Montages

All outputs below are from the course simulator, with a button on D2 (`INPUT_PULLUP`) that bounces for a few milliseconds like a real one.

5.1 Counting presses: three versions

```
// (a) level test: counts while held
if (digitalRead(BTN) == LOW) count++;
```

One 200 ms press gives **1889** in the simulator (thousands on a real board): `loop()` runs thousands of times during one press.

```
// (b) edge detection: counts transitions HIGH -> LOW
int s = digitalRead(BTN);
if (s == LOW && lastState == HIGH) count++;
lastState = s;
```

Two presses give **6**: each press bounces, and every bounce is a new HIGH→LOW edge.

```
// (c) edge detection + debounce: accept a new level only after 50 ms of stability
int reading = digitalRead(BTN);
if (reading != lastReading) { lastChange = millis(); lastReading = reading; }
if (millis() - lastChange >= 50 && reading != stableState) {
  stableState = reading;
  if (stableState == LOW) count++;
}
```

Two presses give **2**. This is the version an exam answer must contain when it says "count the presses".

5.2 Blink without `delay`

```
const unsigned long PERIOD = 500;
unsigned long previous = 0;
int ledState = LOW;
void loop() {
  if (millis() - previous >= PERIOD) {
    previous += PERIOD;
    ledState = !ledState;
    digitalWrite(LED, ledState);
  }
  // other work runs here every pass: buttons, Serial, sensors
}
```

Pin 13 toggles at 500, 1000, 1500, 2000 ms. With `delay(500)` instead, nothing else in `loop()` would run for half a second at a time.

5.3 Toggle an LED on each press

Use pattern (c); on each accepted press: `ledOn = !ledOn; digitalWrite(LED, ledOn ? HIGH : LOW);`. Using (a) makes the LED flicker while the button is held and end in a random state.

5.4 A Serial command interpreter

```
void loop() {
  if (Serial.available() > 0) {
    char cmd = Serial.read();          // 'R', 'G' or 'B'
    int val = Serial.parseInt();       // "R128" -> 128
    val = constrain(val, 0, 255);
    if (cmd == 'R') analogWrite(Rpin, val);
    else if (cmd == 'G') analogWrite(Gpin, val);
    else if (cmd == 'B') analogWrite(Bpin, val);
  }
}
```

Mind the newline bug of Part 1: the trailing `\n` becomes a command of its own on the next pass.

5.5 The FSM skeleton the papers ask you to complete

```
enum class State { RED, GREEN, YELLOW };
State state = State::RED;
unsigned long enteredAt = 0;
void enter(State s) { state = s; enteredAt = millis(); }   // every transition goes through here
void loop() {
  unsigned long t = millis() - enteredAt;                  // time spent in the current state
  switch (state) {
    case State::RED:    setLights(HIGH, LOW, LOW);  if (t >= 5000) enter(State::GREEN);  break;
    case State::GREEN:  setLights(LOW, LOW, HIGH);  if (t >= 4000 || pedestrianPressed()) enter(State::
    YELLOW); break;
    case State::YELLOW: setLights(LOW, HIGH, LOW);  if (t >= 1000) enter(State::RED);    break;
  }
}
```

Outputs depend on the state; transitions depend on time and inputs; no `delay`, so the button is read on every pass. Part G of the complete guide covers the table-to-code method and the audit questions.

6 Part 5: Quiz

One line each; the answer key is on the next page.

1. After `pinMode(8, OUTPUT);` and no `digitalWrite`, pin 8 is: (a) floating (b) LOW (c) HIGH (d) pulled up.
2. A button between D2 and GND, `INPUT_PULLUP`. Pressed, `digitalRead(2)` returns?
3. The same button with `INPUT` and no resistor. Released, it reads?
4. `digitalWrite(4, HIGH)` on a pin configured as `INPUT` does what?
5. `analogRead` of a 2.5 V signal returns about?
6. `analogWrite(9, 64)`: duty cycle and average voltage?
7. `analogWrite(7, 200)` on the Uno does what?
8. List the PWM pins of the Uno.
9. `map(256, 0, 1023, 0, 255)` returns?
10. Type of `millis()` and when does it wrap?
11. Does `if (millis() - last >= 1000)` still work after the wrap? Why?
12. What goes wrong with `int t = millis();`?
13. A button is pressed and released during `delay(5000)`. Is it seen?

14. At 9600 baud, roughly how long does one character take?
15. `Serial.print(66)` versus `Serial.write(66)`?
16. `Serial.read()` with an empty buffer returns?
17. The user types `7` and the code does `int x = Serial.read();`. Value of `x`?
18. `sizeof(int)` on the Uno?
19. `int x = 32767; x = x + 1;` gives?
20. `unsigned long t = 60 * 1000;` gives? How to fix it?
21. `float r = 5 / 2;` gives?
22. `byte b = 250; b += 10;` gives?
23. A function with `static int n = 0; n++; return n;` is called three times. Third return value?
24. EEPROM size and write endurance?
25. Why not put an LED on D0 in a sketch that uses Serial?
26. Series resistor for a 2 V LED at 15 mA from 5 V?
27. Recommended and absolute maximum current per pin?
28. A potentiometer has its ends on 5 V and GND and its wiper on A0. Turned fully to the 5 V end, A0 reads?
29. In the divider 5 V–LDR–A0–10 kΩ–GND, more light makes the reading go up or down?
30. Common-anode RGB LED: `analogWrite(Rpin, 0)` gives?
31. How are two Arduinos wired for Serial?
32. What is bouncing, and two ways to remove it?

Answers: Quiz key

1. (b) LOW: an output starts at 0.
2. LOW: the button connects the pin to GND.
3. Undefined: the pin floats.
4. Turns on the internal pull-up.
5. About 512 ($2.5 \times 1023/5 = 511.5$).
6. $64/255 = 25\,\%$; $5 \times 0.25 \approx 1.25$ V average.
7. Pin 7 has no PWM: values ≥ 128 give a steady HIGH.
8. 3, 5, 6, 9, 10, 11.
9. 63 ($256 \times 255/1023 = 63.8$, truncated).
10. `unsigned long`; after 2^{32} ms ≈ 49.7 days.
11. Yes: unsigned subtraction is modular, the difference is still the elapsed time.
12. `int` holds 32 767 at most: after 32.8 s the value is wrong (negative).
13. No: `delay` blocks, the pin is not read during those 5 s.
14. About 1 ms (10 bits per byte at 9600 bit/s).
15. `66` as text versus the byte 66, shown as `B`.
16. -1.
17. 55, the code of `'7'`; the digit is `x - '0'`.
18. 2 bytes.
19. $-32\,768$.
20. 4 294 961 760 (the `int` product overflows first); write `60000UL` or `60UL * 1000`.
21. 2.00 (integer division happens before the conversion).
22. 4 (260 wraps modulo 256).
23. 3: the static variable keeps its value between calls.
24. 1024 bytes; about 100 000 writes per cell.
25. D0 is RX: Serial and uploading use it.
26. $R = 3/0.015 = 200\ \Omega$; take the standard 220 Ω.
27. 20 mA recommended, 40 mA absolute maximum (200 mA for the chip).
28. 1023.
29. Up: the LDR's resistance falls, so a larger share of 5 V appears across the 10 kΩ.
30. Red at full brightness (the logic is inverted).
31. TX to RX and RX to TX, plus a common GND, same baud rate.
32. One press makes several fast transitions; filter in software (accept a level only after 20–50 ms with `millis()`) or in hardware (a capacitor across the button).